CHENNAI MATHEMATICAL INSTITUTE

Load Balancing in Data Center Networks

by

Hemadri Shekhar Das

A thesis submitted in partial fulfillment for the degree of Masters in Computer Science

under the guidance of

Prof. Y. Chandramouli

May 2026

Declaration of Authorship

I, HEMADRI SHEKHAR DAS, declare that this thesis titled, 'Load Balancing in Data Center Networks' and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a masters degree at this Institute.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: Hemadri Shekhar Das

Date: 24/05/2026

"I had a funny UDP joke to tell, but I lost it somewhere..."

nus-cs2105-blog

Abstract

Load balancing is a fundamental challenge in data centre networks, where efficient traffic distribution is essential for achieving high throughput, low latency, and optimal resource utilization. Equal-Cost Multi-Path (ECMP) routing is widely adopted in modern data centres because of its simplicity, scalability, and low implementation overhead. However, ECMP lacks awareness of real-time network congestion, which can lead to uneven traffic distribution and degraded performance under dynamic traffic conditions. To address these limitations, adaptive congestion-aware approaches such as CONGA have been proposed. CONGA improves traffic distribution by utilizing network performance information to make more responsive routing decisions, though it introduces additional complexity and often depends on ASIC-specific hardware features.

This thesis investigates existing load-balancing approaches in data centre networks, with a particular focus on ECMP and CONGA, through simulation-based evaluation. The first objective is to explore extensions to load-balancing mechanisms that incorporate network performance information in order to improve traffic distribution and overall network efficiency. The second objective is to evaluate whether the performance benefits of CONGA can be achieved without relying on ASIC-specific features, and to examine how its congestion-awareness principles can be integrated into ECMP.

Based on this study, a new load-balancing algorithm is proposed that combines the simplicity and scalability of ECMP with the adaptivity and responsiveness of CONGA. The proposed approach aims to achieve improved network performance while maintaining practical deployability in data centre environments.

Acknowledgements

I would like to express my sincere gratitude to my advisor, Professor Chandramouli, for their constant guidance, support, and encouragement throughout the course of this thesis. Their insights and feedback were invaluable in shaping this work. I am also deeply grateful for their patience in listening to my sometimes random and incoherent thoughts, and for thoughtfully providing feedback that helped turn them into something meaningful.

I am also deeply thankful to Professor Kumar and Professor Madhavan for generously providing compute resources at such short notice, which made it possible to carry out the simulations required for this research. Their timely support significantly contributed to the completion of this thesis.

Finally, and most importantly, I would like to thank my parents, whose patience and unconditional support carried me through this journey — including the long nights where I had entered a feral "debugging mode," engaged in bitter, personal conflict with my compiler, and was no longer fit for normal human interaction.

Contents

Declaration of Authorship i

Abstract iii

Acknowledgements iv

List of Figures viii

List of Tables ix

1 Introduction 1
- 1.1 Motivation . . . 2
- 1.2 Problem Statement . . . 3
- 1.3 Objectives . . . 3
- 1.4 Scope of the Study . . . 3
- 1.5 Organization of the Thesis . . . 4

2 Background Theory 6
- 2.1 Common Load Balancing Algorithms . . . 6
  - 2.1.1 Static Methods . . . 6
  - 2.1.2 Dynamic Methods . . . 7
- 2.2 ECMP . . . 7
  - 2.2.1 Equal-Cost Multi-Path (ECMP) . . . 7
  - 2.2.2 Important Complexity Result . . . 8
  - 2.2.3 When ECMP Works Well . . . 9
- 2.3 Power of Two Choices (P2C) . . . 9
  - 2.3.1 Working Principle . . . 9
  - 2.3.2 Mathematical Results . . . 9
- 2.4 CONGA . . . 10
  - 2.4.1 Working Principle . . . 10
  - 2.4.2 Load Balancing Decision . . . 10
- 2.5 Comparison . . . 11
  - 2.5.1 Architectural Differences . . . 11
  - 2.5.2 Computational Complexity . . . 11

2.5.2.1 ECMP Complexity . . . 12
2.5.2.2 P2C Complexity . . . 12
2.5.2.3 CONGA Complexity . . . 13
2.5.2.4 Key Insight . . . 14
2.6 Our Target . . . 14

3 Our Algorithm **16**
3.1 Challenges to Solve . . . 16
3.2 Algorithm Overview . . . 17
3.3 Detailed Algorithm Description . . . 17
3.4 Computational Complexity Analysis . . . 18
3.5 Conclusion . . . 19

4 Experimental Setup **20**
4.1 Choice of Simulator . . . 20
4.2 Realistic Traffic Generator . . . 20
4.2.1 Choice of Traffic Generator . . . 21
4.3 A Provable Baseline . . . 21
4.4 Complete Experimental Setup . . . 22

5 Experiment 1 **23**
5.1 Standard Per-Flow ECMP . . . 23
5.2 CONGA . . . 23
5.3 Results . . . 24
5.3.1 Raw Data Comparison . . . 24
5.3.1.1 Raw Flow Completion Time(FCT) Comparisons . . . 24
5.3.1.2 FCT Normalised to ECMP . . . 25
5.3.2 The Results Match the Theory . . . 26
5.4 Conclusions . . . 27

6 Experiment 2 **28**
6.1 Results . . . 28
6.1.1 Raw Data Comparison . . . 28
6.1.1.1 Raw Flow Completion Time(FCT) Comparisons . . . 28
6.1.1.2 FCT Normalised to ECMP . . . 30
6.2 Expected Behaviour . . . 31
6.3 Conclusion . . . 31

7 Experiment 3 **32**
7.1 Modifications to CONGA . . . 32
7.2 Results . . . 33
7.2.0.1 FCT Normalised to ECMP . . . 34
7.3 A Probable Explanation . . . 35
7.4 Conclusion . . . 36

8 Experiment 4 **37**
8.1 Implementation Approach . . . 37
8.2 RTT Measurement Challenges . . . 37

8.3 Design Decision and Final Approach 38

9 Results and Conclusions **39**

10 Future Work **41**

Bibliography **43**

List of Figures

5.1 Commercial Cloud 24
5.2 Private Enterprise 24
5.3 Social Media Cloud 25
5.4 University 25
5.5 Commercial Cloud 25
5.6 Private Enterprise 26
5.7 Social Media Cloud 26
5.8 University 26

6.1 Commercial Cloud 28
6.2 Private Enterprise 29
6.3 Social Media Cloud 29
6.4 University 29
6.5 Commercial Cloud 30
6.6 Private Enterprise 30
6.7 Social Media Cloud 30
6.8 University 31

7.1 Commercial Cloud 33
7.2 Private Enterprise 33
7.3 Social Media Cloud 33
7.4 University 34
7.5 Commercial Cloud 34
7.6 Private Enterprise 34
7.7 Social Media Cloud 35
7.8 University 35

List of Tables

2.1 Advantages and Limitations of ECMP 8
2.2 Advantages and Limitations of P2C 10
2.3 Advantages and Limitations of CONGA 11
2.4 Architectural comparison of ECMP, P2C, and CONGA 12

Chapter 1

Introduction

In today's digital world, where staying connected is no longer a luxury but a necessity and an integral part of everyday life, data centers serve as the foundational infrastructure powering the global Internet. A data center is defined as a facility containing various types of equipment such as servers, storage devices, and networking hardware connected using carefully designed network topologies. Collectively, this equipment processes, stores, and transmits digital information and is commonly referred to as Information Technology (IT) infrastructure.

Data Center Networks (DCNs) are designed differently from traditional enterprise or home networks. They operate at massive scale, require extremely low latency, and must provide high reliability and high bandwidth. In such environments, network congestion can significantly degrade performance. Some links may become overloaded while others remain underutilized, leading to packet loss, increased latency, and poor throughput.

To address these challenges, load balancing techniques are widely used and play an important role in proper functioning of the datacenter. Load balancing distributes network traffic evenly across multiple servers or communication paths so that no single resource becomes a bottleneck. Efficient load balancing improves bandwidth utilization, reduces congestion, minimizes response time, and enhances fault tolerance.

This chapter introduces the motivation behind studying load balancing in data center networks, defines the problem statement, presents the objectives and scope of this thesis, and outlines the organization of the remaining chapters.

1.1 Motivation

Several load balancing algorithms are commonly implemented in modern data centers depending on their use cases and deployment requirements. Over the years, many additional algorithms have been proposed, each introducing improvements in scalability, adaptability, or congestion awareness.

Among the most widely used algorithms are Round Robin, Least Connections, Power of Two Choices (P2C), and perhaps the most dominant load balancing mechanism in modern data center routing, Equal-Cost Multi-Path (ECMP).

However, it is well known that ECMP can often balance traffic poorly [1–7]. Since ECMP distributes flows using hash-based path selection, hash collisions may cause multiple large flows to be mapped onto the same path, creating severe traffic imbalance. More importantly, ECMP makes purely local forwarding decisions without considering downstream congestion conditions. As a result, ECMP performs poorly under asymmetric network conditions caused by dynamic traffic patterns or link failures, which occur frequently in data center environments [3, 8].

To overcome these limitations, several advanced load balancing algorithms specifically designed for data center networks have been proposed. Some of the most notable among them are Flare [1], Hedera [7], TinyFlow [9], MPTCP [2], and one of the most influential congestion-aware load balancing algorithms, CONGA [10].

Although these algorithms improve traffic distribution in certain scenarios, each of them has limitations that restrict large-scale practical deployment.

Among these algorithms, CONGA is particularly important because it has been implemented in real production data center hardware. CONGA improves upon ECMP by incorporating congestion-aware routing decisions using distributed feedback mechanisms.

However, CONGA also suffers from several drawbacks. First, under normal operating conditions where no significant congestion or failures exist, CONGA may perform worse than ECMP because its computational overhead is significantly higher than that of ECMP. While ECMP performs simple hash-based forwarding, CONGA continuously monitors and evaluates congestion metrics, introducing additional processing overhead.

Second, and perhaps more importantly, CONGA requires custom ASIC support for VXLAN encapsulation and decapsulation in order to operate at line speed. This dependence on specialized hardware limits its deployability and increases implementation complexity and cost.

These limitations motivate the need for a new approach that combines the simplicity and efficiency of ECMP with the congestion-awareness of CONGA while avoiding dependence on specialized hardware.

1.2 Problem Statement

Based on the limitations of existing approaches, the problem addressed in this thesis can be formulated as follows:

> Can the performance of CONGA be improved to outperform ECMP under normal operating conditions, or can the ASIC dependence of CONGA be eliminated, or alternatively, can the congestion-awareness principles of CONGA be incorporated into ECMP to achieve better overall performance?

1.3 Objectives

The objectives of this thesis are to study the structure and behavior of modern data center networks, analyze existing load balancing mechanisms such as ECMP, P2C, and CONGA, and evaluate their computational and practical limitations. The work further aims to implement these algorithms in a simulation environment using ns-3 and validate their performance under realistic traffic conditions.

In addition, the thesis investigates modifications to CONGA in order to improve its efficiency and reduce overhead. Based on these findings, a new load balancing approach is designed that combines the simplicity of ECMP with congestion-awareness principles inspired by CONGA. Finally, the proposed approach is evaluated in terms of throughput, latency, congestion handling, and computational overhead using realistic simulation workloads.

1.4 Scope of the Study

In this thesis, we first analyze the computational complexity of both ECMP and CONGA and explain why CONGA may perform worse than ECMP under normal network conditions.

Next, we explore modifications to CONGA in an attempt to improve its performance and reduce its overhead.

Based on the observations obtained from these analyses, we then propose a new load balancing algorithm that combines ECMP with a global congestion metric inspired by CONGA. Unlike CONGA, the proposed approach avoids dependence on custom ASIC hardware and is designed to be implementable using modern programmable networking technologies such as P4 or eBPF.

The evaluation of the proposed algorithm is carried out using realistic traffic simulations in ns-3, focusing on metrics such as throughput, latency, congestion handling, and computational overhead.

1.5 Organization of the Thesis

The remainder of this thesis is structured to progressively build the necessary background, methodology, and evaluation of the proposed work. It begins in Chapter 2 with a detailed presentation of the theoretical foundations of load balancing in data center networks. This chapter introduces the core concepts underlying modern data center architectures and examines widely used load balancing techniques, including ECMP, P2C, and CONGA, with emphasis on their operational principles and inherent limitations.

Chapter 3 then introduces the proposed load balancing approach developed in this work. It discusses the motivation for its design, outlines the system architecture, and provides a formal analysis of its computational complexity and design trade-offs in comparison to existing methods.

The experimental methodology used to evaluate the proposed system is described in Chapter 4. This includes a detailed explanation of the ns-3 simulation environment, the traffic generation framework, the network topology used for evaluation, and the baseline configurations employed for comparative analysis.

Chapter 5 focuses on the initial set of experimental results, where the performance of ECMP and CONGA is evaluated under a range of realistic traffic workloads. Building on this, Chapter 6 examines modifications made to CONGA, analyzing their impact on performance and highlighting the limitations observed in these variations.

Further experimental evaluations and implementation-specific details of the proposed algorithm are presented in Chapter 7, including extended scenarios designed to test robustness and scalability under varying network conditions.

Chapter 8 brings together all experimental observations into a unified discussion, identifying key trends and insights that emerge across different workloads and system configurations.

Finally, Chapter 9 concludes the thesis by summarizing the main contributions of the work and outlining potential directions for future research.

Chapter 2

Background Theory

In this chapter, we discuss the fundamental challenges involved in load balancing within datacenter networks. We compare commonly used load balancing algorithms with CONGA and establish the design objectives that motivate the proposed work in this thesis.

2.1 Common Load Balancing Algorithms

Load balancing in datacenter networks relies on a range of algorithmic strategies to distribute incoming traffic across multiple servers or network paths. These strategies range from simple static approaches that do not consider runtime conditions to more advanced dynamic methods that adapt based on observed system state.

2.1.1 Static Methods

One of the simplest load balancing strategies is Round Robin, where incoming requests are distributed sequentially across available servers. This approach is computationally inexpensive and easy to implement, but it assumes homogeneous server capacity and does not account for variations in workload or network conditions. As a result, it can lead to imbalance when request sizes or service times are highly variable.

Another widely used static approach is Equal-Cost Multi-Path (ECMP) routing. In ECMP, traffic is distributed across multiple paths that have equal routing cost. The selection is typically performed using a hash computed over packet header fields such as source IP, destination IP, ports, and protocol information. While ECMP ensures

flow consistency and avoids packet reordering, it does not incorporate any real-time information about network congestion or link utilization.

2.1.2 Dynamic Methods

Dynamic load balancing techniques attempt to improve upon static methods by incorporating runtime system information. The Least Connections method routes traffic to the server with the fewest active connections, thereby improving load distribution in environments where request durations vary significantly. However, this method assumes that the number of active connections is a reliable proxy for load, which may not always hold in heterogeneous systems.

The Power of Two Choices (P2C) is another lightweight dynamic strategy. Instead of selecting a single server deterministically, P2C randomly samples two candidates and forwards the request to the one with the lower observed load. This approach significantly improves load distribution while maintaining low computational overhead, making it scalable for large distributed systems.

2.2 ECMP

2.2.1 Equal-Cost Multi-Path (ECMP)

Equal-Cost Multi-Path (ECMP) is a widely deployed routing technique used in datacenter and enterprise networks to distribute traffic across multiple equal-cost paths. It improves bandwidth utilization by enabling parallel forwarding across multiple network paths while maintaining per-flow consistency.

In ECMP, traffic is distributed across N equal-cost paths using a deterministic hashing function. The hash is typically computed from packet header fields such as source and destination IP addresses, ports, and protocol type:

$$\texttt{hash} = \texttt{CRC32}(\texttt{srcIP}, \texttt{dstIP}, \texttt{srcPort}, \texttt{dstPort}, \texttt{protocol}) \tag{2.1}$$

The selected forwarding path is then determined as:

$$\texttt{path_index} = \texttt{hash} \bmod N \tag{2.2}$$

This ensures that all packets belonging to the same flow are consistently mapped to the same path, thereby preventing packet reordering while maintaining simplicity in implementation.

Advantages	**Limitations**
Stable and predictable flow-level routing	Ignores flow size and traffic demand
Low computational overhead	Hash collisions may lead to imbalance
Hardware-supported in modern switches	No awareness of congestion or link utilization
Simple deployment and configuration	Cannot adapt to dynamic traffic conditions
Scales well in large topologies	Large flows may dominate individual paths
Provides automatic path diversity under failures	Limited to flow-level granularity

TABLE 2.1: Advantages and Limitations of ECMP

2.2.2 Important Complexity Result

Despite its practical advantages, ECMP has a fundamental limitation in terms of optimal traffic allocation. It has been shown in prior work [11] that computing optimal ECMP weight assignments can be computationally intractable in general network topologies.

The network can be modeled as a graph:

$$G = (V, E), \quad c_e = \text{capacity}, \quad d_{st} = \text{demand} \tag{2.3}$$

The objective is to minimize the maximum link utilization:

$$\min \max_{e \in E} \frac{f_e}{c_e} \tag{2.4}$$

If ECMP is interpreted as a mapping from link weights to shortest-path routing and flow splitting, then determining an optimal weight configuration becomes computationally hard due to reductions from classical NP-complete problems such as 3-SAT and Partition.

This implies that globally optimal ECMP configurations are generally not feasible to compute in arbitrary topologies.

2.2.3 When ECMP Works Well

This complexity result, however, applies primarily to arbitrary and irregular network topologies. In structured datacenter architectures such as Fat-Tree and Leaf-Spine (Clos) networks, multiple equal-cost paths naturally exist between hosts. In such cases, ECMP is able to distribute traffic relatively uniformly across available paths, resulting in good load balancing performance and high scalability.

2.3 Power of Two Choices (P2C)

The Power of Two Choices (P2C) is a simple yet powerful probabilistic load balancing technique widely used in distributed systems. Unlike ECMP, which relies on static hashing, P2C introduces limited dynamic decision-making by sampling multiple candidates and selecting the least loaded option.

2.3.1 Working Principle

P2C operates by randomly selecting two candidate servers or paths and forwarding the request to the one with the lower observed load. Although the sampling process is simple, it significantly improves load distribution compared to purely random assignment. This is because even minimal comparison introduces global imbalance correction over time.

The load metric used in P2C can vary depending on the system and may include queue length, RTT, or link utilization. The key requirement is that the metric must reflect current system state with reasonable accuracy.

2.3.2 Mathematical Results

The effectiveness of P2C can be understood using the classical balls-and-bins model. If n balls are placed uniformly at random into n bins, the maximum load grows as:

$$\frac{\log n}{\log \log n} \tag{2.5}$$

However, if each ball is assigned by choosing $d \geq 2$ random bins and selecting the least loaded one, the maximum load improves significantly and becomes:

$$\frac{\log \log n}{\log d} + O(1) \tag{2.6}$$

This demonstrates that even a small amount of additional choice leads to a dramatic reduction in load imbalance.

Advantages	**Limitations**
Low computational overhead	Requires real-time load estimation
Improved load distribution	Performance depends on metric accuracy
Scales well in large systems	Metrics can become stale under high load
Simple and practical implementation	Slightly more complex than ECMP
Near-optimal balancing behavior	Requires additional comparison step

TABLE 2.2: Advantages and Limitations of P2C

2.4 CONGA

CONGA [10] (Congestion-aware Next Generation Load Balancing) is a distributed load balancing mechanism designed specifically for datacenter networks. It improves performance by dynamically adapting routing decisions based on real-time congestion information.

2.4.1 Working Principle

CONGA divides traffic into smaller units called flowlets [1], allowing fine-grained routing decisions without causing packet reordering. Switches continuously estimate congestion levels across available paths and exchange this information between leaf switches.

Based on this feedback, each flowlet is assigned to the path with the lowest observed congestion. This allows CONGA to react quickly to changes in traffic conditions and achieve better load distribution compared to static hashing approaches.

CONGA is typically implemented in custom ASIC hardware and is optimized for leaf-spine datacenter topologies where multiple parallel paths exist between endpoints.

2.4.2 Load Balancing Decision

In CONGA, only the first packet of a flowlet triggers a routing decision. Subsequent packets follow the same path until a new flowlet is detected. This reduces control overhead while maintaining stability.

A path is selected based on minimizing the combined local and remote congestion estimate:

$$p = \arg\min_{p} \max(\text{Local DRE}, \text{Remote Congestion}) \tag{2.7}$$

where Local DRE represents locally observed congestion and Remote Congestion represents feedback received from other switches in the network.

Advantages	**Limitations**
Fast adaptation to congestion	Requires specialized switch hardware
High accuracy congestion-aware routing	Higher implementation complexity
Efficient handling of asymmetric traffic	Limited deployment outside datacenters
Flowlet-based stability	Additional state and control overhead
Near-optimal performance in Clos networks	Hard to deploy in general networks

Table 2.3: Advantages and Limitations of CONGA

2.5 Comparison

2.5.1 Architectural Differences

The three approaches differ significantly in their design philosophy, decision location, and state requirements. ECMP operates entirely using stateless hashing at routers, while P2C introduces limited host-level decision-making using sampled load information. In contrast, CONGA relies on distributed congestion state maintained across switches in the network fabric.

2.5.2 Computational Complexity

Although ECMP, P2C, and CONGA differ significantly in design, state requirements, and congestion-awareness mechanisms, all three exhibit approximately linear scalability with respect to the number of processed packets. However, this similarity in asymptotic behavior hides important differences in how cost is distributed across packets, flowlets, and control-state operations.

Feature	**ECMP**	**P2C**	**CONGA**
Decision location	Router-based hashing	Host or dispatcher	Leaf-spine switches
State requirement	Stateless	Partial local state	Distributed congestion state
Load information	None	Local sampling	Global congestion feedback
Control mechanism	Deterministic hashing	Random sampling + comparison	Flowlet-based adaptive routing
Adaptivity	None	Moderate	High
Granularity	Flow-level	Flow-level	Flowlet-level
Congestion awareness	None	Partial	Full
Complexity	Low	Low-Medium	High

TABLE 2.4: Architectural comparison of ECMP, P2C, and CONGA

2.5.2.1 ECMP Complexity

ECMP performs load balancing using a deterministic hash computed over packet header fields. For each packet, the operations consist of hash computation, modulo selection, and forwarding table lookup. All of these operations are constant time.

Thus, the per-packet complexity is:

$$T_{\text{ECMP, packet}} = O(1) \tag{2.8}$$

For a total of P packets, the overall complexity is:

$$T_{\text{ECMP}} = O(P) \tag{2.9}$$

ECMP therefore has minimal computational overhead and no dependence on network state or traffic dynamics.

2.5.2.2 P2C Complexity

The Power of Two Choices (P2C) introduces lightweight dynamic decision-making by sampling two candidate paths and selecting the one with the lower observed load. Each request involves random sampling, retrieval of two load metrics, and a constant-time comparison.

Therefore, the per-request complexity is:

$$T_{\text{P2C, request}} = O(1) \tag{2.10}$$

For P requests, the total complexity becomes:

$$T_{\text{P2C}} = O(P) \tag{2.11}$$

Although asymptotically equivalent to ECMP, P2C introduces additional runtime overhead due to load metric collection, which increases constant factors in practical deployments.

2.5.2.3 CONGA Complexity

CONGA introduces significantly richer control logic through flowlet-based routing and distributed congestion awareness. Let P be the number of packets, m be the number of flowlets, and n be the number of available uplinks.

For each flowlet, CONGA evaluates congestion across all candidate paths, resulting in a cost of:

$$T_{\text{flowlet}} = O(n) \tag{2.12}$$

Across all flowlets, this becomes:

$$T_{\text{flowlet total}} = O(mn) \tag{2.13}$$

In addition, each packet incurs constant-time operations for forwarding, flowlet tracking, and congestion state updates:

$$T_{\text{packet}} = O(P) \tag{2.14}$$

Therefore, the total complexity is:

$$T_{\text{CONGA}} = O(mn + P) \tag{2.15}$$

Why CONGA Reduces to $O(P)$ in Datacenters Although CONGA appears more expensive than ECMP and P2C, its behavior in practical datacenter environments is closer to linear. This is due to two key properties of datacenter traffic.

First, the number of candidate paths n is small and fixed by the topology (e.g., leaf-spine networks), meaning n does not scale with network size. Second, traffic in datacenters is typically dominated by long-lived elephant flows interspersed with short mice flows.

Elephant flows generate a large number of packets per flowlet, which amortizes the flowlet-level decision cost over many packets. In contrast, mice flows generate very few packets, but their small size means they contribute minimally to the overall system-wide computational cost. As a result, the total number of flowlets m grows much more slowly than the total packet count P, typically satisfying $m \ll P$ in practice.

Under these conditions, the dominant term becomes packet processing rather than flowlet evaluation, leading to the practical approximation:

$$T_{\text{CONGA}} \approx O(P) \tag{2.16}$$

However, unlike ECMP and P2C, CONGA still carries significantly higher constant-factor overhead due to congestion state maintenance, flowlet detection, and distributed feedback exchange between switches. This overhead is especially noticeable in mice-flow dominated workloads where packet counts are low but decision logic is still invoked.

2.5.2.4 Key Insight

While ECMP, P2C, and CONGA all achieve linear scaling with respect to packet count, their practical performance differs due to differences in constant factors, state maintenance, and congestion-awareness mechanisms.

This relationship can be summarized as:

$$T_{\text{ECMP}} < T_{\text{P2C}} < T_{\text{CONGA}} \tag{2.17}$$

where the inequality reflects implementation overhead rather than asymptotic complexity.

2.6 Our Target

The primary objective of this thesis is to bridge the gap between ECMP and CONGA in terms of performance while maintaining practicality and ease of deployment. Specifically,

the goal is to either enhance ECMP with lightweight congestion-awareness mechanisms or simplify CONGA to reduce its operational overhead.

In both cases, the focus is on achieving near-CONGA performance with ECMP-level simplicity, thereby enabling scalable and deployable load balancing solutions for modern datacenter networks.

Chapter 3

Our Algorithm

We initially attempted to combine the Power of Two Choices (P2C) with CONGA in order to reduce computational overhead while preserving congestion-awareness. However, experimental observations showed that the performance degraded compared to baseline CONGA.

Our reasoning is that CONGA already maintains a highly accurate congestion metric. Therefore, selecting any path other than the best path according to this metric introduces unnecessary suboptimal routing decisions, reducing overall performance. In essence, CONGA's strength lies in consistently following fine-grained congestion feedback, and restricting this decision process weakens its effectiveness.

As a result, we shifted our focus toward improving ECMP while retaining its simplicity and deployability.

3.1 Challenges to Solve

Compared to CONGA, ECMP has several inherent limitations in datacenter environments. It does not handle link failures and asymmetric congestion effectively since routing decisions are based on static hashing rather than real-time network state. It is also susceptible to hash collisions, which can lead to uneven traffic distribution across available paths. Furthermore, ECMP does not explicitly prevent scenarios where short latency-sensitive flows are delayed due to congestion caused by long-lived high-throughput flows.

CONGA addresses these limitations using global congestion awareness, congestion-based path selection, and flowlet-level routing, which together enable more adaptive and fine-grained load balancing.

3.2 Algorithm Overview

Our proposed approach can be interpreted as a Weighted ECMP scheme that uses RTT as a lightweight proxy for global congestion information. The key idea is to continuously estimate per-path congestion using RTT measurements and use these estimates for probabilistic routing decisions.

The first step is RTT estimation. We use TCP timestamps to measure round-trip time as defined in RFC 7323 [12]. To reduce noise in measurements, we apply an Exponentially Weighted Moving Average (EWMA) filter to smooth RTT values over time.

In the initialization phase, when no prior measurements exist for a path, a baseline RTT value is assumed. This ensures that all paths are initially treated equally and avoids bias during early routing decisions.

Path selection is then performed based on the current RTT estimates. Paths with lower RTT values are assigned higher probability of selection, either through weighted probabilistic selection or a power-of-two style comparison mechanism.

After each transmission, RTT estimates are updated using newly observed samples. This feedback loop allows the algorithm to dynamically adapt to changing network conditions.

Flowlet detection is used to ensure stability and avoid packet reordering. For each packet, the inter-packet arrival gap is computed, and if this gap exceeds the estimated RTT of the current path, a new flowlet is assumed. At this point, path selection is reconsidered.

3.3 Detailed Algorithm Description

Let $r_1, r_2, \ldots, r_n$ denote the EWMA-smoothed RTT estimates for n candidate paths. The algorithm first computes the total RTT across all paths as

$$R = \sum_{i=1}^{n} r_i \tag{3.1}$$

To convert RTT values into selection scores, we define inverse-based weights as

$$s_i = R - r_i \tag{3.2}$$

The total weight mass is then computed as

$$S = \sum_{i=1}^{n} s_i \tag{3.3}$$

Finally, the probability of selecting path i is given by

$$p_i = \frac{s_i}{S} \tag{3.4}$$

This formulation ensures that lower RTT paths receive higher selection probability, while still maintaining a non-zero probability for all paths. This prevents starvation and improves robustness under dynamic network conditions.

The intuition behind this design is that

$$p_i \propto \frac{1}{r_i} \tag{3.5}$$

which approximates inverse-RTT-based routing while remaining simple to implement.

3.4 Computational Complexity Analysis

We analyze the complexity in terms of number of packets P, number of flowlets m, and number of candidate paths n.

For each flowlet, the algorithm evaluates all candidate paths and computes normalization, resulting in a cost of $O(n)$ per flowlet. Across m flowlets, this contributes $O(mn)$.

For each packet, constant-time operations are performed for hashing, RTT estimation, and EWMA updates. This contributes $O(P)$ overall.

Therefore, the total computational complexity is

$$T(P) = O(mn + P) \tag{3.6}$$

In typical datacenter environments, the number of paths n is small and fixed by topology, so the algorithm scales approximately linearly:

$$T(P) \approx O(P) \tag{3.7}$$

However, compared to ECMP, the constant factor is higher due to RTT tracking, EWMA computation, and flowlet detection overhead.

3.5 Conclusion

At first glance, the computational complexity of the proposed algorithm appears similar to CONGA. However, the key distinction lies in deployment practicality.

CONGA requires specialized hardware support for maintaining and propagating congestion information in real time. In contrast, the proposed approach can be implemented using existing programmable networking frameworks such as P4 or eBPF without requiring custom ASIC support.

Therefore, even when performance is comparable to CONGA, the proposed method offers advantages in terms of deployability, flexibility, and compatibility with existing software-defined networking systems.

Chapter 4

Experimental Setup

4.1 Choice of Simulator

For our experiments, we chose the **ns-3** simulator.

ns-3 is a discrete-event network simulator designed for Internet systems research and educational use. It is open-source software released under the GNU GPLv2 license and maintained by a large research community. ns-3 provides detailed packet-level simulation, making it well suited for evaluating datacenter networking algorithms such as ECMP, CONGA, and our proposed approach.

One of the key advantages of ns-3 is that it allows precise control over network behavior at the packet level while still preserving realistic protocol interactions. This makes it particularly useful for studying congestion-sensitive mechanisms, where small changes in queueing or timing can significantly affect performance.

In addition, ns-3 provides accurate network protocol implementations, fine-grained timing and queue modelling, support for custom routing and congestion-control mechanisms, and scalability for large network topologies. These features make ns-3 an appropriate platform for evaluating congestion-aware load-balancing schemes.

4.2 Realistic Traffic Generator

Modern datacenter load-balancing algorithms are highly optimized for realistic traffic patterns; therefore, evaluating them under synthetic uniform traffic is insufficient and may lead to misleading conclusions.

However, obtaining real production datacenter traces is difficult due to privacy and infrastructure constraints. To address this limitation, we used **TrafPy** [13], a traffic generation framework designed specifically for datacenter network simulation.

TrafPy generates realistic traffic workloads by modeling mice and elephant flow distributions, burstiness and temporal locality, application-level communication patterns, and variable flow arrival processes. These properties are important because real datacenter workloads are highly heterogeneous, with a mixture of short latency-sensitive flows and long bandwidth-intensive flows occurring simultaneously.

By incorporating these characteristics, TrafPy enables the simulation to better approximate real-world datacenter traffic behavior, rather than relying on idealized or uniform traffic assumptions.

4.2.1 Choice of Traffic Generator

TrafPy was selected for several practical and methodological reasons. It supports realistic datacenter traffic distributions and is compatible with packet-level simulation workflows used in ns-3. It also allows flexible and configurable workload generation, which is necessary for evaluating algorithms under different load conditions. In addition, it supports large-scale traffic matrix generation, making it suitable for emulating datacenter-style communication patterns. Finally, it provides standard datacenter workload distributions, which helps in ensuring comparability with prior work.

In particular, TrafPy enables the generation of workloads containing both mice flows, which are short-lived and latency-sensitive, and elephant flows, which are long-lived and throughput-intensive. This distinction is especially important because congestion-aware algorithms such as CONGA and our proposed approach respond differently depending on whether the network is dominated by short or long flows, and therefore must be evaluated under both conditions to obtain meaningful results.

4.3 A Provable Baseline

To establish a reliable baseline for comparison, we first reproduced simulations for both ECMP and CONGA.

The objective of this step was to ensure that our simulation environment produces behavior consistent with the results reported in the original CONGA paper. Matching this baseline is important because even small deviations in implementation details, such

as queueing behavior or hashing strategy, can significantly impact performance results in datacenter simulations. Once the baseline behavior aligned with published observations, we used these implementations as reference points for evaluating our proposed approach.

This validation step ensures that any observed improvements or degradations in later experiments are attributable to the algorithmic changes rather than inconsistencies or artifacts in the simulation environment.

4.4 Complete Experimental Setup

For the CONGA implementation, we initially used an existing GitHub repository [14]. However, the implementation was developed for an older version of ns-3 and required several modifications to function correctly in our environment.

To ensure compatibility and reproducibility, we executed the entire simulation environment inside an Ubuntu 18.04 Docker container. This approach also helped isolate dependencies and avoid inconsistencies across different systems. During setup, we identified and fixed multiple issues in the CONGA implementation through code inspection, debugging, and iterative testing.

Our network topology consists of 2 spine switches, 2 leaf switches, and 2 hosts connected to each leaf switch, resulting in a total of 4 hosts arranged in a standard leaf-spine datacenter architecture. This topology, while small in scale, is sufficient to capture the effects of load balancing decisions across multiple equal-cost paths.

For traffic generation, we used standard workload traces provided by TrafPy, including University workloads, Private enterprise workloads, Social media cloud workloads, and Commercial cloud workloads. These workloads were selected to represent a diverse range of datacenter traffic patterns, allowing us to evaluate algorithm performance under different levels of burstiness, flow size distribution, and communication intensity.

Chapter 5

Experiment 1

5.1 Standard Per-Flow ECMP

In modern datacenter networks, **per-flow** ECMP is the standard load-balancing approach because it avoids packet reordering while still distributing traffic across multiple equal-cost paths.

However, the default ECMP implementation provided in ns-3 uses **per-packet** forwarding. Since per-packet ECMP can introduce packet reordering, it does not accurately represent real production datacenter behavior. Moreover, the original CONGA paper compares against **per-flow** ECMP rather than per-packet ECMP. Using the default ns-3 implementation could therefore lead to unfair or misleading comparisons. To ensure consistency with prior work and realistic deployment scenarios, we implemented our own per-flow ECMP mechanism within ns-3.

This implementation hashes each flow onto a single path and ensures that all packets belonging to the same flow follow the same route throughout the simulation.

5.2 CONGA

For the CONGA implementation, we used a modified version of the implementation provided in [14].

The original codebase was designed for an older version of ns-3 and required several modifications to function correctly in our experimental environment. In addition to compatibility fixes, we also corrected multiple implementation issues identified during debugging and testing.

5.3 Results

Both algorithms were evaluated under varying theoretical network loads ranging from 10% to 90%.

In this experiment, all links were assumed to be symmetric. Therefore, the topology does not contain any intentional link asymmetry or bandwidth imbalance.

5.3.1 Raw Data Comparison

5.3.1.1 Raw Flow Completion Time(FCT) Comparisons

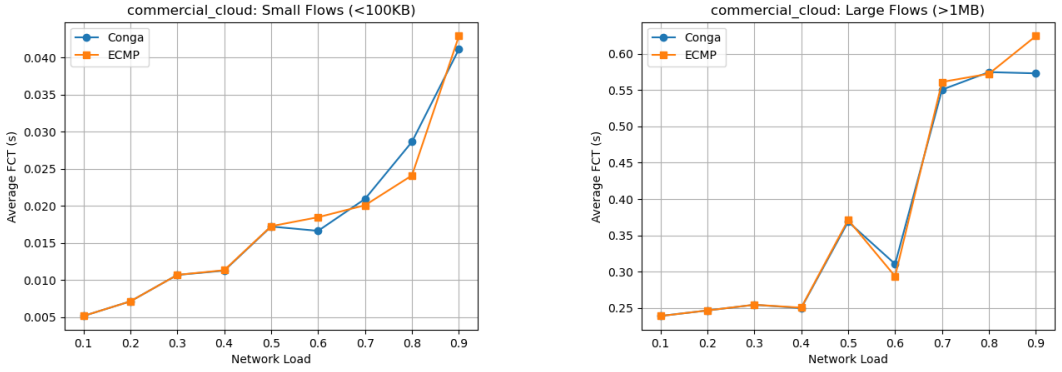


FIGURE 5.1: Commercial Cloud

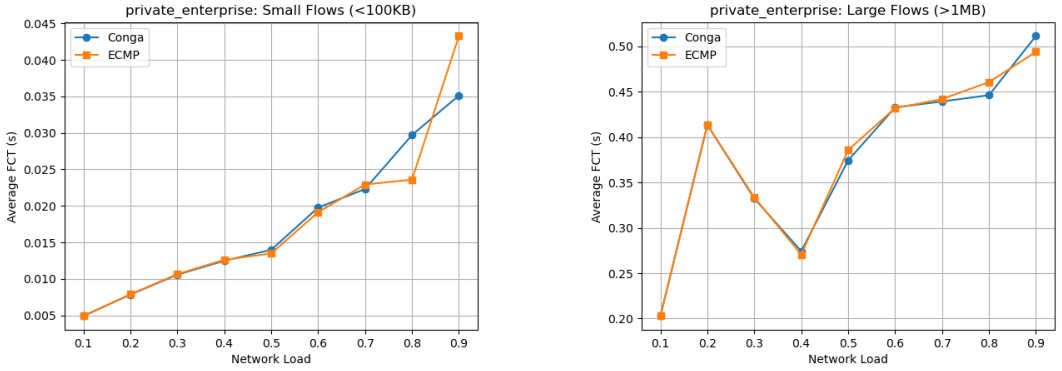


FIGURE 5.2: Private Enterprise

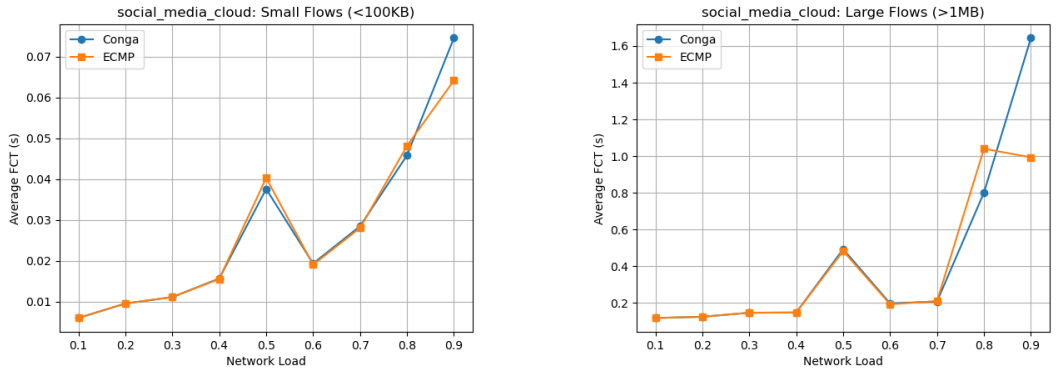


FIGURE 5.3: Social Media Cloud

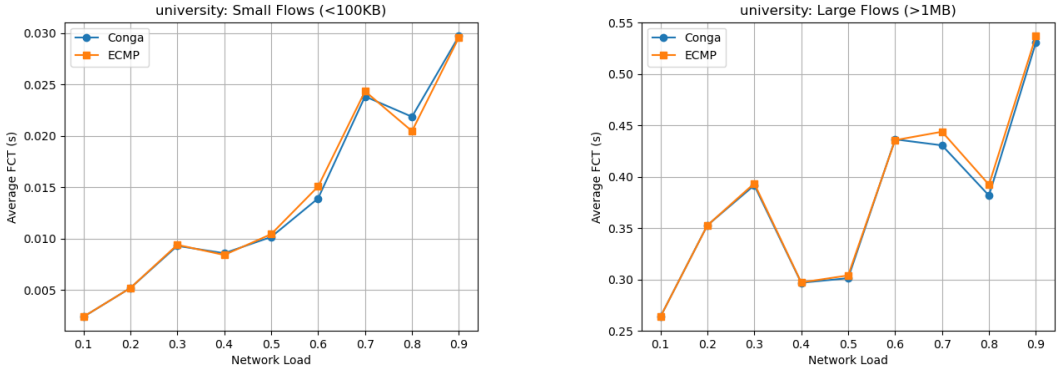


FIGURE 5.4: University

5.3.1.2 FCT Normalised to ECMP

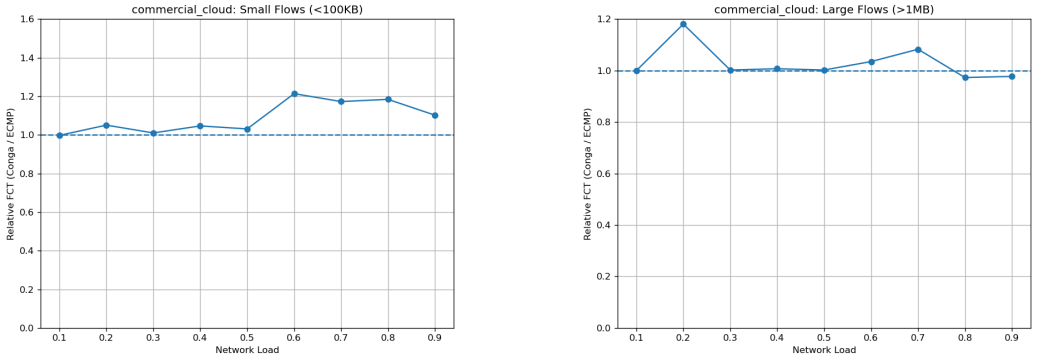


FIGURE 5.5: Commercial Cloud

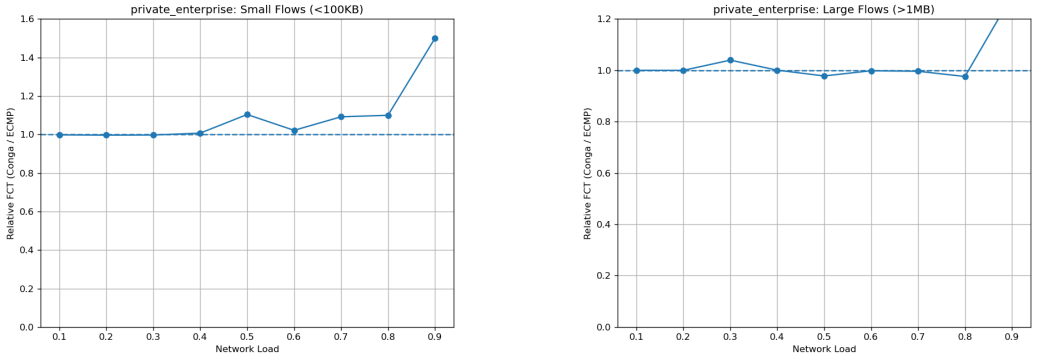


FIGURE 5.6: Private Enterprise

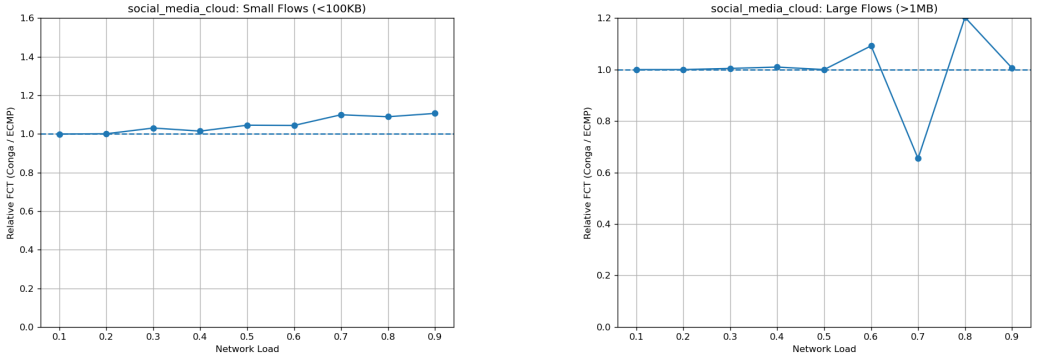


FIGURE 5.7: Social Media Cloud

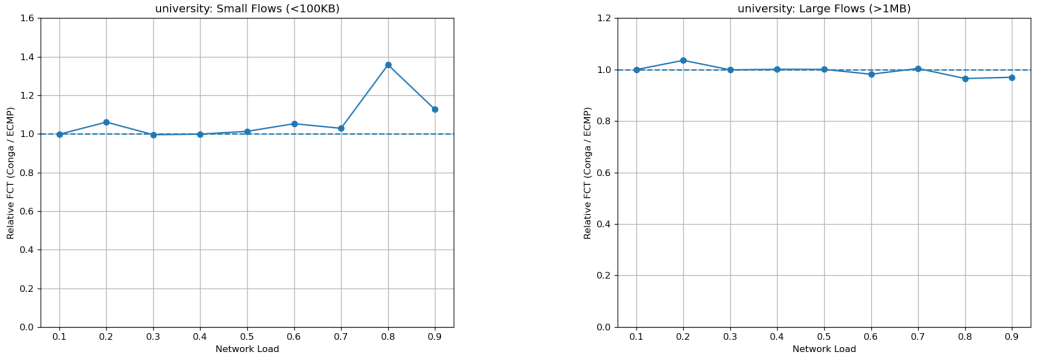


FIGURE 5.8: University

5.3.2 The Results Match the Theory

Across all workload types, the observed results are broadly consistent with the theoretical behavior predicted by our earlier complexity analysis of ECMP and CONGA.

Mice Flows For mice flows, ECMP performs significantly better than CONGA, particularly at higher network loads. This behavior is mainly due to the additional computation and congestion-feedback overhead introduced by CONGA.

Since mice flows are short-lived and latency-sensitive, the extra processing involved in congestion-aware routing can become a noticeable fraction of the total flow completion time. In contrast, ECMP uses a much simpler forwarding strategy with lower decision overhead, allowing it to handle short flows more efficiently in our setup.

Elephant Flows For elephant flows, the performance gap between ECMP and CONGA becomes much smaller across all traffic distributions.

Unlike mice flows, elephant flows are dominated more by sustained throughput and congestion behavior rather than routing computation overhead alone. As a result, the adaptive congestion-aware routing used by CONGA helps offset its higher complexity by improving traffic distribution across available paths.

This trend closely matches the conclusions drawn from our earlier complexity analysis of ECMP and CONGA. The experimental observations therefore support the theoretical expectation that ECMP benefits from lower routing overhead, making it favorable for short-lived flows, while CONGA's congestion-awareness becomes more useful for larger flows where network congestion dominates overall performance.

5.4 Conclusions

Overall, our experimental results are consistent with both our complexity analysis and the performance trends reported in the original CONGA paper. In particular, we observe similar relative behavior between ECMP and CONGA under varying traffic loads and flow types.

These results support the correctness of our implementation and indicate that the simulation environment faithfully reproduces expected datacenter networking behavior.

Chapter 6

Experiment 2

In this experiment, we repeat the same evaluation setup as in Experiment 1, but introduce a single random failure in one of the dual aggregate links connecting the leaf and spine switches. This allows us to study how ECMP and CONGA behave under partial network failure conditions.

6.1 Results

6.1.1 Raw Data Comparison

6.1.1.1 Raw Flow Completion Time(FCT) Comparisons

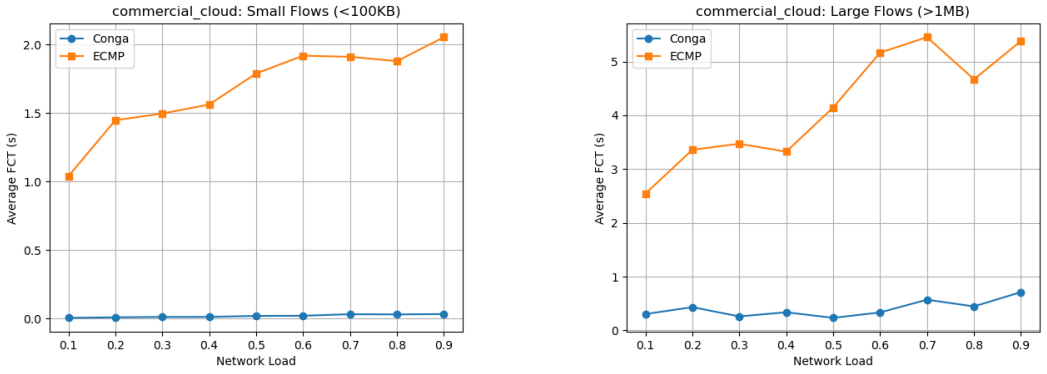


FIGURE 6.1: Commercial Cloud

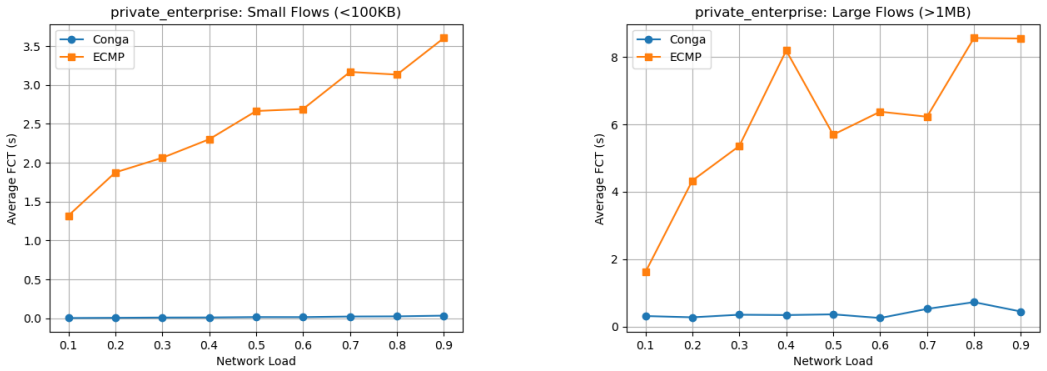


FIGURE 6.2: Private Enterprise

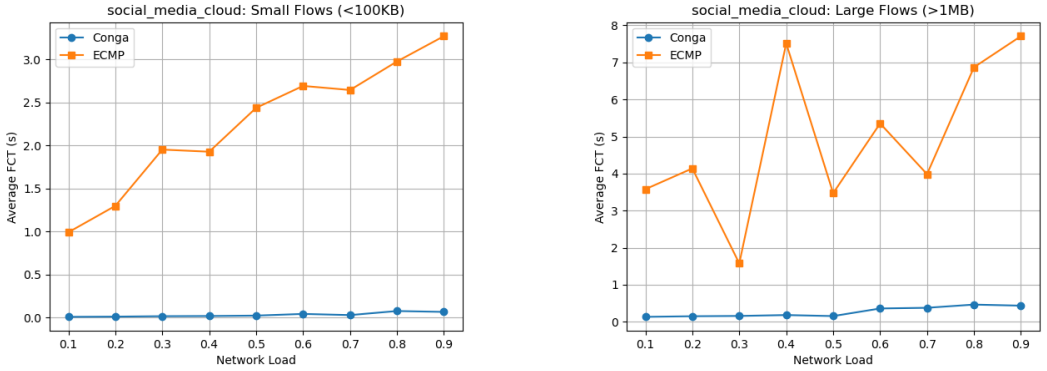


FIGURE 6.3: Social Media Cloud

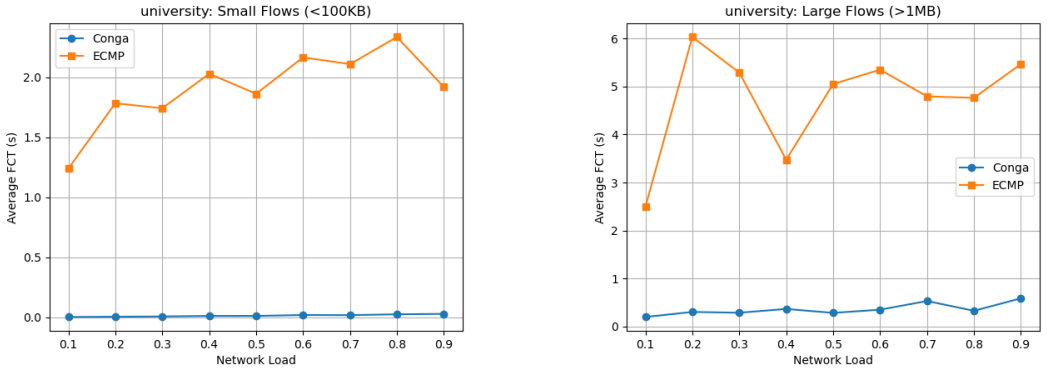


FIGURE 6.4: University

6.1.1.2 FCT Normalised to ECMP

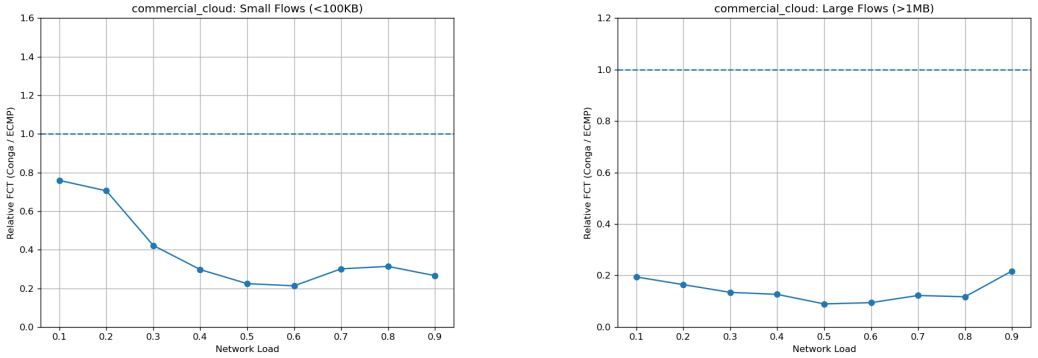


FIGURE 6.5: Commercial Cloud

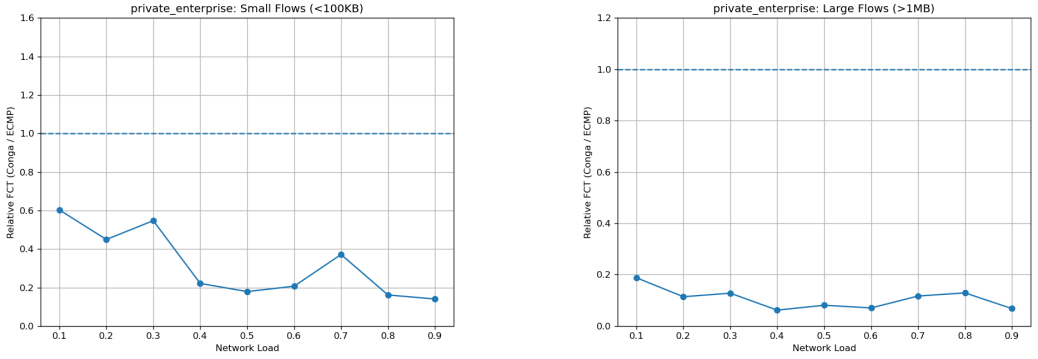


FIGURE 6.6: Private Enterprise

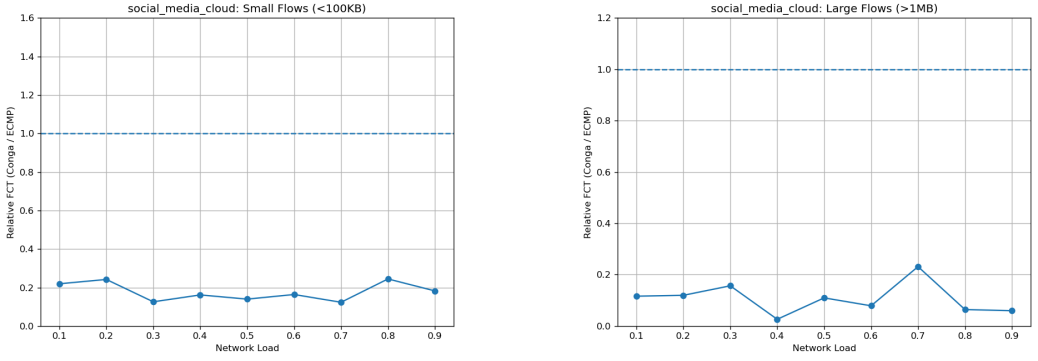


FIGURE 6.7: Social Media Cloud

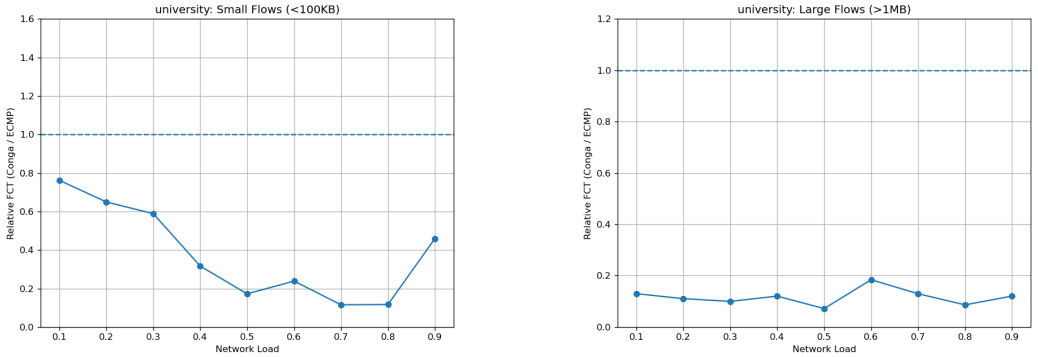


FIGURE 6.8: University

6.2 Expected Behaviour

As expected, CONGA performs better in asymmetric or failure-prone scenarios due to its use of global congestion information for routing decisions. In contrast, ECMP continues to rely on static hashing and is unable to adapt to changes in available path capacity, which leads to suboptimal load distribution under link failures.

6.3 Conclusion

This experiment further validates the reproducibility of our simulation setup and confirms that our implementation aligns with the expected behavior reported in the original CONGA study.

Chapter 7

Experiment 3

In this experiment, we attempt to design an improved load-balancing mechanism by modifying CONGA.

7.1 Modifications to CONGA

We explore the idea of incorporating the Power of two Choices(P2C) into CONGA, with the goal of reducing its computational overhead while maintaining or improving performance. The intuition is that a simpler decision process could make CONGA more competitive with ECMP in terms of efficiency.

However, the results were not as expected.

We evaluate three variants in total. The first variant, v1/base, corresponds to the baseline CONGA implementation. In the second variant, v2, two paths are selected at random and the one with the lower congestion metric is chosen. In the third variant, v3, the two least-congested paths are first identified, and then one of them is selected at random.

7.2 Results

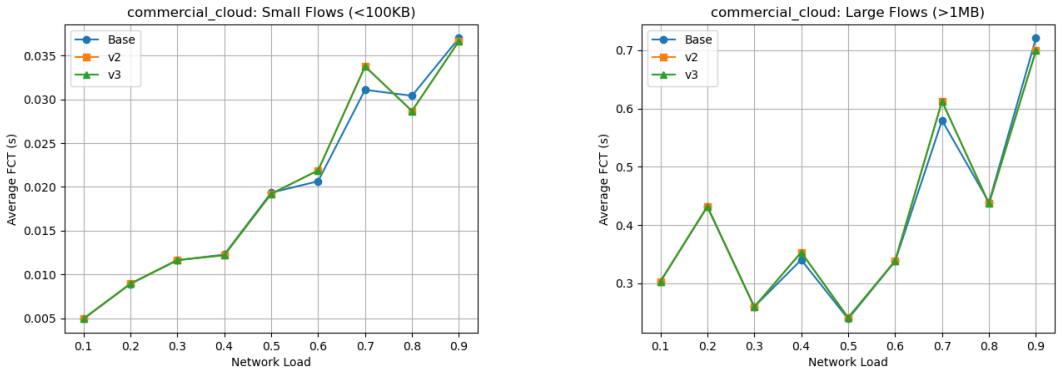


FIGURE 7.1: Commercial Cloud

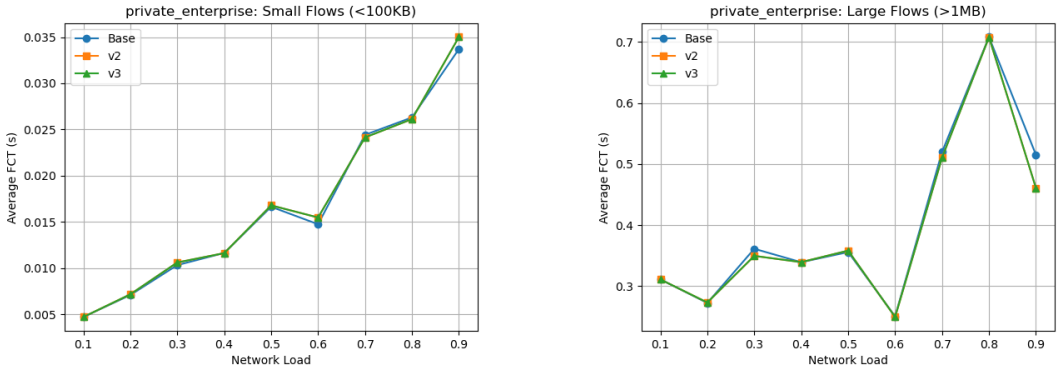


FIGURE 7.2: Private Enterprise

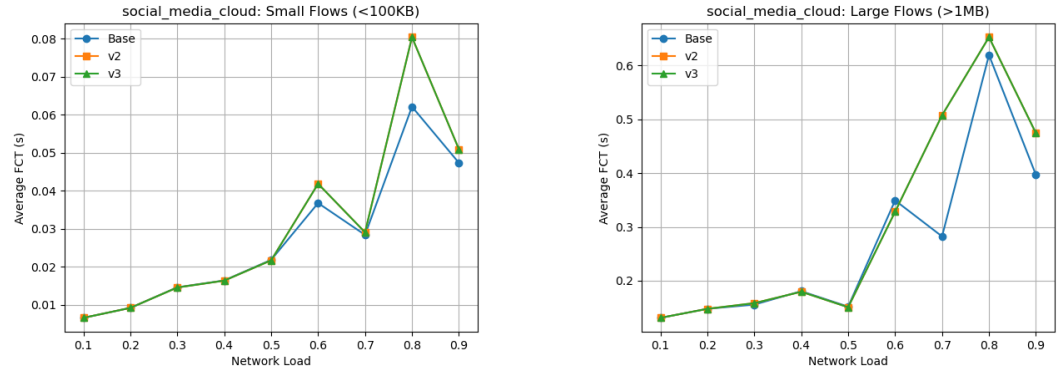


FIGURE 7.3: Social Media Cloud

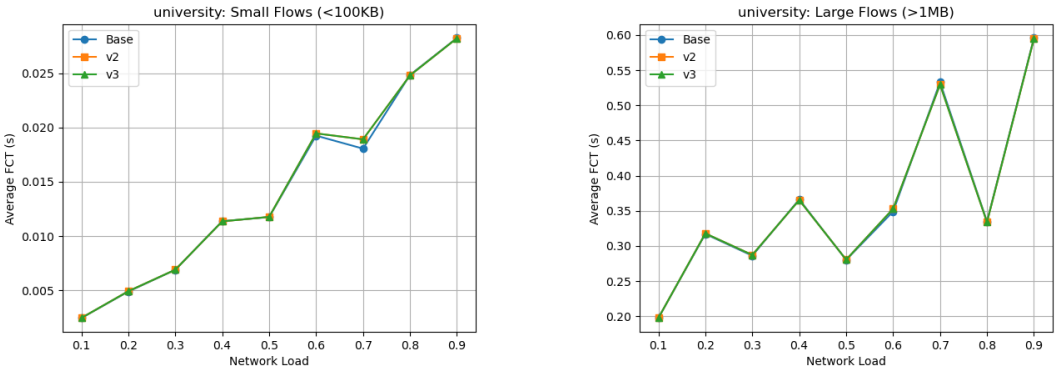


FIGURE 7.4: University

7.2.0.1 FCT Normalised to ECMP

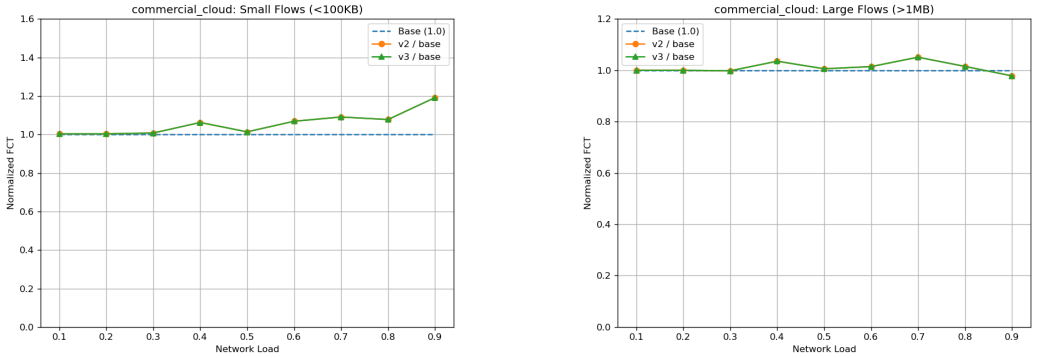


FIGURE 7.5: Commercial Cloud

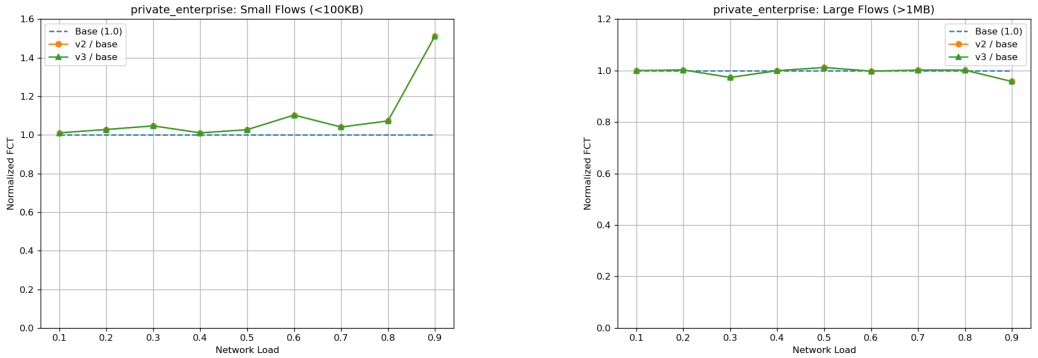


FIGURE 7.6: Private Enterprise

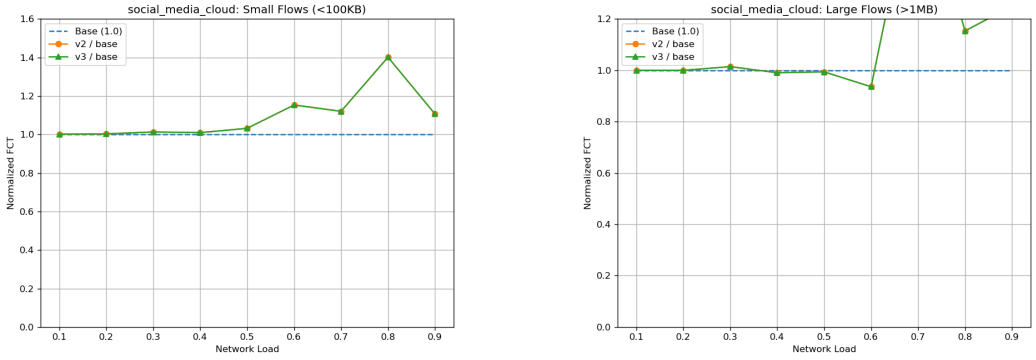


FIGURE 7.7: Social Media Cloud

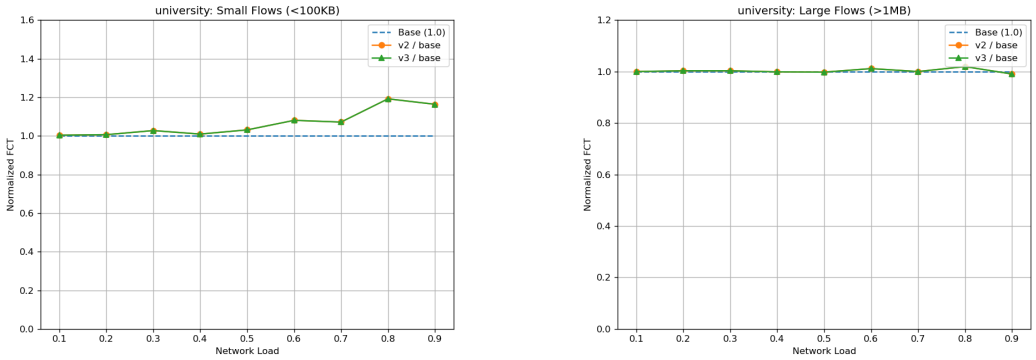


FIGURE 7.8: University

7.3 A Probable Explanation

In our experimental setup, the topology consists of two spine and two leaf switches. This implies that, for any given flow, only two distinct path choices are available. As a result, the behavior of both v2 and v3 effectively collapses to the same decision process, which explains why their performance curves overlap almost perfectly.

We initially intended to evaluate these variants under larger and more diverse topologies with greater path diversity. However, due to time constraints, we were limited to this smaller configuration.

Despite this limitation, a consistent observation can still be made. Both v2 and v3 perform worse than the baseline CONGA across workloads and load conditions.

A plausible explanation for this behavior is that CONGA's congestion metric is already highly effective at identifying the optimal path under the given conditions. Any deviation from always selecting the best-ranked path introduces either randomness or deliberate

suboptimal selection, which can only maintain or degrade performance relative to the baseline. In this sense, the additional decision mechanisms introduced in v2 and v3 do not provide useful diversity beyond what CONGA's congestion-aware routing already captures.

7.4 Conclusion

The results suggest that, when using the same congestion metric as CONGA, there is limited scope for improvement through modifications in path selection alone. Any meaningful improvement is likely to require changes in how congestion information is measured or propagated, rather than altering the final selection strategy on top of an already strong metric.

Chapter 8

Experiment 4

This experiment was originally intended to evaluate the performance of the proposed algorithm in a complete simulation setting. However, due to implementation challenges and time constraints, it was not possible to fully complete the system integration and experimental validation. As a result, this chapter focuses on the practical difficulties encountered during the implementation phase, particularly those related to integrating the algorithm into the ns-3 simulation framework.

8.1 Implementation Approach

In ns-3, custom routing and load balancing mechanisms are typically implemented by extending the IPv4 routing module. This requires developing new routing logic that integrates with the existing packet forwarding pipeline while preserving compatibility with the simulator's internal networking stack.

Although this process is straightforward in principle, it introduces a significant level of implementation complexity when working with flowlet-based routing and dynamic path selection. In particular, careful integration is required to ensure that routing updates do not interfere with existing protocol behavior or simulation correctness.

8.2 RTT Measurement Challenges

A key component of the proposed algorithm is accurate RTT estimation using TCP timestamps, as defined in RFC 7323. While ns-3 provides support for TCP timestamp options, the available implementation is primarily designed for flow-level measurements and not for flowlet-level path-aware tracking.

This introduces a fundamental challenge, as flowlets in the proposed system may traverse multiple paths during their lifetime. However, ns-3 does not directly associate RTT measurements with the specific path taken at the time of transmission. As a result, additional mechanisms are required to maintain consistency between measured RTT values and the actual network path used.

One possible approach is to maintain a per-flowlet state table that tracks the currently active path for each flowlet and updates it dynamically as routing decisions change. However, this introduces two practical difficulties. First, the size of the tracking table may grow continuously during simulation, requiring a well-defined eviction or cleanup policy to prevent unbounded memory usage. Determining an appropriate strategy for removing stale entries without affecting correctness is non-trivial. Second, when a flowlet changes its path, there is a risk that RTT samples may still correspond to packets that were transmitted on a previous path. This can result in inconsistent or misleading RTT estimates, particularly due to delayed acknowledgments, which may significantly affect measurement accuracy.

An alternative approach is to directly extract TCP timestamps from packet headers within ns-3. However, this requires modifications to the packet processing pipeline and increases implementation complexity, as it involves handling low-level packet traversal logic within the simulator. Such modifications also introduce additional points of failure and increase the risk of unintended interactions with existing protocol behavior.

8.3 Design Decision and Final Approach

To avoid these complications, a more controlled approach was adopted using ns-3 packet tags for timestamp propagation. Packet tags provide a lightweight mechanism for attaching metadata to packets within the simulation environment without modifying the underlying protocol stack.

Although this approach does not reflect real-world TCP header behavior directly, it is consistent with common practices in ns-3-based congestion control and load balancing research, where similar abstractions are used to simplify simulation of hardware-level operations. In particular, the implementations of CONGA we are using also relies on this rather than full protocol-level encapsulation and decapsulation.

This approach was therefore selected as the most stable and implementation-feasible solution, allowing RTT estimation to be performed consistently while maintaining simulation correctness and avoiding unnecessary complexity in the packet processing pipeline.

Chapter 9

Results and Conclusions

This chapter summarizes the key findings from the experimental evaluation of ECMP, CONGA, and the modified load balancing methods proposed and tested in this thesis.

Overall, the results support the main observations from the original CONGA paper. Specifically, CONGA performs comparably to ECMP under normal conditions when the network is balanced, and there are no significant congestion or link failures. In these cases, ECMP often does slightly better or at least matches CONGA, mainly due to its lower computing requirements and simpler forwarding logic.

However, in asymmetric situations like uneven traffic distribution or link failures, CONGA consistently outperforms ECMP. This improvement comes from CONGA's ability to use distributed congestion feedback for better routing decisions. By adjusting to congestion across multiple paths, CONGA avoids overloaded links and achieves a better overall load distribution. This leads to improved flow completion times and less congestion in stressed network environments.

In addition to validating CONGA's behavior, this thesis explored possible improvements by adding the Power of Two Choices (P2C) mechanism into CONGA. The idea behind this change was to reduce decision complexity while keeping congestion-awareness by limiting path selection to a small number of candidate routes. However, the experimental results showed that this change did not enhance performance. In fact, in most cases, the modified versions performed either similarly or worse than the standard CONGA.

This outcome suggests that CONGA's congestion-aware decision mechanism, especially its use of the Discount Rate Estimator (DRE), already offers a highly effective estimation of the network state. The DRE captures congestion conditions in enough detail so that narrowing the decision process to a smaller candidate set, like in the P2C approach, removes useful global information that helps in optimal path selection. As a result,

simplifying the decision process in this way does not improve performance and may actually worsen it due to suboptimal choices.

These findings indicate that the design choices in CONGA are well-suited to its target deployment environment in leaf-spine data center networks. While there is still potential for reducing implementation costs or improving ease of deployment, the key congestion-aware mechanism already appears to be close to optimal within its design framework.

In conclusion, the experiments show that CONGA has clear advantages in asymmetric and failure-prone situations, while ECMP remains competitive in stable and low-congestion environments. Furthermore, efforts to improve CONGA through P2C-based simplifications did not produce better results, reinforcing the effectiveness of its original congestion estimation method.

Chapter 10

Future Work

The immediate direction for future work is the complete implementation and evaluation of the proposed load balancing algorithm introduced in this thesis. While the current work provides a theoretical design and complexity analysis, a full experimental validation against ECMP and CONGA is necessary to establish its practical effectiveness.

Since the proposed approach introduces probabilistic elements in path selection, it is inherently non-deterministic in nature. Unlike deterministic schemes such as ECMP or the structured congestion feedback mechanism used in CONGA, the behavior of the algorithm may vary across different runs under identical conditions. Therefore, a thorough evaluation is required across a wide range of scenarios, including varying traffic loads, flow size distributions, and network topologies. Such extensive testing is essential to ensure that the observed performance trends are consistent, statistically significant, and not artifacts of specific simulation runs.

Beyond direct evaluation, there are several promising directions for extending this work. One particularly interesting direction is the integration of reinforcement learning (RL) techniques into the path selection process. In this formulation, the routing problem can be treated as a sequential decision-making task, where the system continuously learns from observed network feedback such as RTT variations, congestion signals, and flow completion performance.

In such an RL-based extension, the model could be trained to predict optimal path selection decisions based on historical and real-time network state information. Instead of directly using instantaneous RTT values or congestion metrics, packets or flowlets could query a trained policy model that encodes long-term knowledge of network behavior. This would potentially allow the system to move beyond reactive decision-making and toward a more predictive routing strategy.

An additional advantage of this approach is the possibility of learning recurring traffic patterns in data center environments. Modern data center workloads often exhibit temporal locality and predictable bursts. A reinforcement learning model, if properly designed and trained, may be able to anticipate periods of high congestion before they fully manifest in the network. This predictive capability could enable proactive load balancing decisions, improving overall network stability and reducing congestion during peak traffic periods.

However, such an extension also introduces several challenges, including training stability, convergence guarantees, and deployment complexity in real-time systems. These issues would need to be carefully addressed in future research before practical adoption.

Overall, reinforcement learning provides a promising direction for evolving congestion-aware load balancing from heuristic-based decision making toward adaptive and data-driven network optimization.

Bibliography

[1] Srikanth Kandula, Dina Katabi, Shantanu Sinha, and Arthur Berger. Dynamic load balancing without packet reordering. *SIGCOMM Comput. Commun. Rev.*, 37(2):51–62, March 2007. ISSN 0146-4833. doi: 10.1145/1232919.1232925. URL https://doi.org/10.1145/1232919.1232925.

[2] Costin Raiciu, Sebastien Barre, Christopher Pluntke, Adam Greenhalgh, Damon Wischik, and Mark Handley. Improving datacenter performance and robustness with multipath tcp. In *Proceedings of the ACM SIGCOMM 2011 Conference*, SIGCOMM '11, page 266–277, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450307970. doi: 10.1145/2018436.2018467. URL https://doi.org/10.1145/2018436.2018467.

[3] Costin Raiciu, Sebastien Barre, Christopher Pluntke, Adam Greenhalgh, Damon Wischik, and Mark Handley. Improving datacenter performance and robustness with multipath tcp. *SIGCOMM Comput. Commun. Rev.*, 41(4):266–277, August 2011. ISSN 0146-4833. doi: 10.1145/2043164.2018467. URL https://doi.org/10.1145/2043164.2018467.

[4] Theophilus Benson, Ashok Anand, Aditya Akella, and Ming Zhang. Microte: fine grained traffic engineering for data centers. In *Proceedings of the Seventh COnference on Emerging Networking EXperiments and Technologies*, CoNEXT '11, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450310413. doi: 10.1145/2079296.2079304. URL https://doi.org/10.1145/2079296.2079304.

[5] Jiaxin Cao, Rui Xia, Pengkun Yang, Chuanxiong Guo, Guohan Lu, Lihua Yuan, Yixin Zheng, Haitao Wu, Yongqiang Xiong, and Dave Maltz. Per-packet load-balanced, low-latency routing for clos-based data center networks. In *Proceedings of the Ninth ACM Conference on Emerging Networking Experiments and Technologies*, CoNEXT '13, page 49–60, New York, NY, USA, 2013. Association for Computing Machinery. ISBN 9781450321013. doi: 10.1145/2535372.2535375. URL https://doi.org/10.1145/2535372.2535375.

[6] Siddhartha Sen, David Shue, Sunghwan Ihm, and Michael J. Freedman. Scalable, optimal flow routing in datacenters via local link balancing. In *Proceedings of the Ninth ACM Conference on Emerging Networking Experiments and Technologies*, CoNEXT '13, page 151–162, New York, NY, USA, 2013. Association for Computing Machinery. ISBN 9781450321013. doi: 10.1145/2535372.2535397. URL https://doi.org/10.1145/2535372.2535397.

[7] Mohammad Al-Fares, Sivasankar Radhakrishnan, Barath Raghavan, Nelson Huang, and Amin Vahdat. Hedera: dynamic flow scheduling for data center networks. In *Proceedings of the 7th USENIX Conference on Networked Systems Design and Implementation*, NSDI'10, page 19, USA, 2010. USENIX Association.

[8] Vincent Liu, Daniel Halperin, Arvind Krishnamurthy, and Thomas Anderson. F10: a fault-tolerant engineered network. In *Proceedings of the 10th USENIX Conference on Networked Systems Design and Implementation*, nsdi'13, page 399–412, USA, 2013. USENIX Association.

[9] Hong Xu and Baochun Li. TinyFlow: Breaking elephants down into mice in data center networks. In *2014 IEEE 20th International Workshop on Local & Metropolitan Area Networks (LANMAN)*, page 3. IEEE, May 2014. doi: 10.1109/LANMAN.2014.7028620.

[10] Mohammad Alizadeh, Abdul Kabbani, Amar Mudipalli, Balaji Vamanan, Hongyi Zeng, Prateek Yalamanchili, Sergey Legtchenko, and David A. Maltz. Conga: Distributed congestion-aware load balancing for datacenters. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, pages 503–514, 2014.

[11] Marco Chiesa, Guy Kindler, and Michael Schapira. Traffic engineering with equal-cost-multipath: An algorithmic perspective. In *IEEE INFOCOM 2014 - IEEE Conference on Computer Communications*, pages 1590–1598, 2014. doi: 10.1109/INFOCOM.2014.6848095.

[12] David Borman, Robert T. Braden, Van Jacobson, and Richard Scheffenegger. TCP Extensions for High Performance. RFC 7323, September 2014. URL https://www.rfc-editor.org/info/rfc7323.

[13] Christopher W.F. Parsonson, Joshua L. Benjamin, and Georgios Zervas. Traffic generation for benchmarking data centre networks. *Optical Switching and Networking*, 46:100695, November 2022. ISSN 1573-4277. doi: 10.1016/j.osn.2022.100695. URL http://dx.doi.org/10.1016/j.osn.2022.100695.

[14] snowzjx. ns3-load-balance. URL https://github.com/snowzjx/ns3-load-balance.